Solution: Nth Digit

Problem Statement

Given a positive integer `n`, find the **nth** digit of the infinite integer sequence `1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13...` and so on.

Examples

- **Example 1**:
  - **Input**: `n = 3`
  - **Expected Output**: `3`
  - **Justification**: The third digit in the sequence `1, 2, 3, 4, 5, 6, 7...` is `3`.
- **Example 2**:
  - **Input**: `n = 110000`
  - **Expected Output**: `2`
  - **Justification**: In the sequence, the 110,000th digit is part of a larger number where the pattern has expanded beyond single, double, and triple digits. Specifically, it falls within the range of five or more digit numbers, landing on a `2`.
- **Example 3**:
  - **Input**: `n = 15`
  - **Expected Output**: `2`
  - **Justification**: In the `1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13...` sequence, the 15th digit is `2`.

Solution

To solve this problem, we first understand the pattern of digit distribution within the sequence. The sequence starts with single-digit numbers (1 to 9), followed by double digits (10 to 99), then triple digits (100 to 999), and so on. This pattern reveals that the number of digits in each group increases as we move further along the sequence. Our approach leverages this pattern to determine the digit's exact location. We'll calculate the range in which the `n`th digit falls by considering the digit lengths and the counts of numbers within those lengths.

This method is effective because it breaks down the problem into manageable segments, allowing us to bypass counting each digit individually. By calculating the total digits within each range and subtracting these from `n`, we can narrow down to the exact number and its digit of interest. This strategic approach reduces the computational complexity, making it a swift solution for finding the `n`th digit in such an extensive sequence.

Step-by-step Algorithm

- Initialize variables to track the length of the digits (`digitLength`), the starting point of numbers within that digit length (`start`), and the count of digits covered by numbers with that digit length (`count`).
- Loop to find the digit length where the `n`th digit falls. For each iteration:
  - Check if `n` is greater than the product of `digitLength` and `count` (the total number of digits covered by numbers of `digitLength` digits). If `n` is greater, subtract this product from `n`, indicating we move to the next range of numbers with longer digit lengths.
  - Update `digitLength` to consider numbers with one more digit.
  - Multiply `count` with 10 to get the count of all digits for numbers with the new `digitLength`.
  - Multiply `start` with 10 get the first number with the current `digitLength`.
- Once the loop ends, the actual number (`targetNumber`) containing the `n`th digit is calculated by adjusting `start` based on `n` and `digitLength`.
- Calculate the exact digit within `targetNumber` that corresponds to `n`.

Algorithm Walkthrough

Let's take the input `n = 110000`.

Initial Values

- `n = 110000`
- `digitLength = 1` (We start considering numbers with one digit.)
- `count = 9` (There are 9 digits in the sequence from 1 to 9.)
- `start = 1` (The sequence starts with the number 1.)

Step 1: Determine Digit Length

- For the first range (1-9), `n > digitLength ∗ count` is `110000 > 1 ∗ 9` (True)
  - Update `n: n = n − (digitLength ∗ count) = 110000 − 9 = 109991`
  - Increment `digitLength: digitLength = 2`
  - Update `count: count = 9 ∗ 10 = 90` (Now considering numbers with two digits.)
  - Update `start: start = 1 ∗ 10 = 10`

Step 2: Move to Two-Digit Numbers

- For the two-digit range (10-99), `n > digitLength ∗ count` is `109991 > 2 ∗ 90` (True)
  - Update `n: n = 109991 − 180 = 109811`
  - Increment `digitLength: digitLength = 3`
  - Update `count: count = 90 ∗ 10 = 900` (Now considering numbers with three digits.)
  - Update `start: start = 10 ∗ 10 = 100`

Step 3: Move to Three-Digit Numbers

- For the three-digit range (100-999), `n > digitLength ∗ count` is `109811 > 3 ∗ 900` (True)
  - Update `n: n = 109811 − 2700 = 107111`
  - Increment `digitLength: digitLength = 4`
  - Update `count: count = 900 ∗ 10 = 9000` (Now considering numbers with four digits.)
  - Update `start: start = 100 ∗ 10 = 1000`

Step 4: Move to Four-Digit Numbers

- For the four-digit range (1000-9999), `n > digitLength ∗ count` is `107111 > 4 ∗ 9000` (True)
  - Update `n: n = 107111 − 36000 = 71111`
  - Increment `digitLength: digitLength = 5`
  - Update `count: count = 9000 ∗ 10 = 90000` (Now considering numbers with five digits.)
  - Update `start: start = 1000 ∗ 10 = 10000`

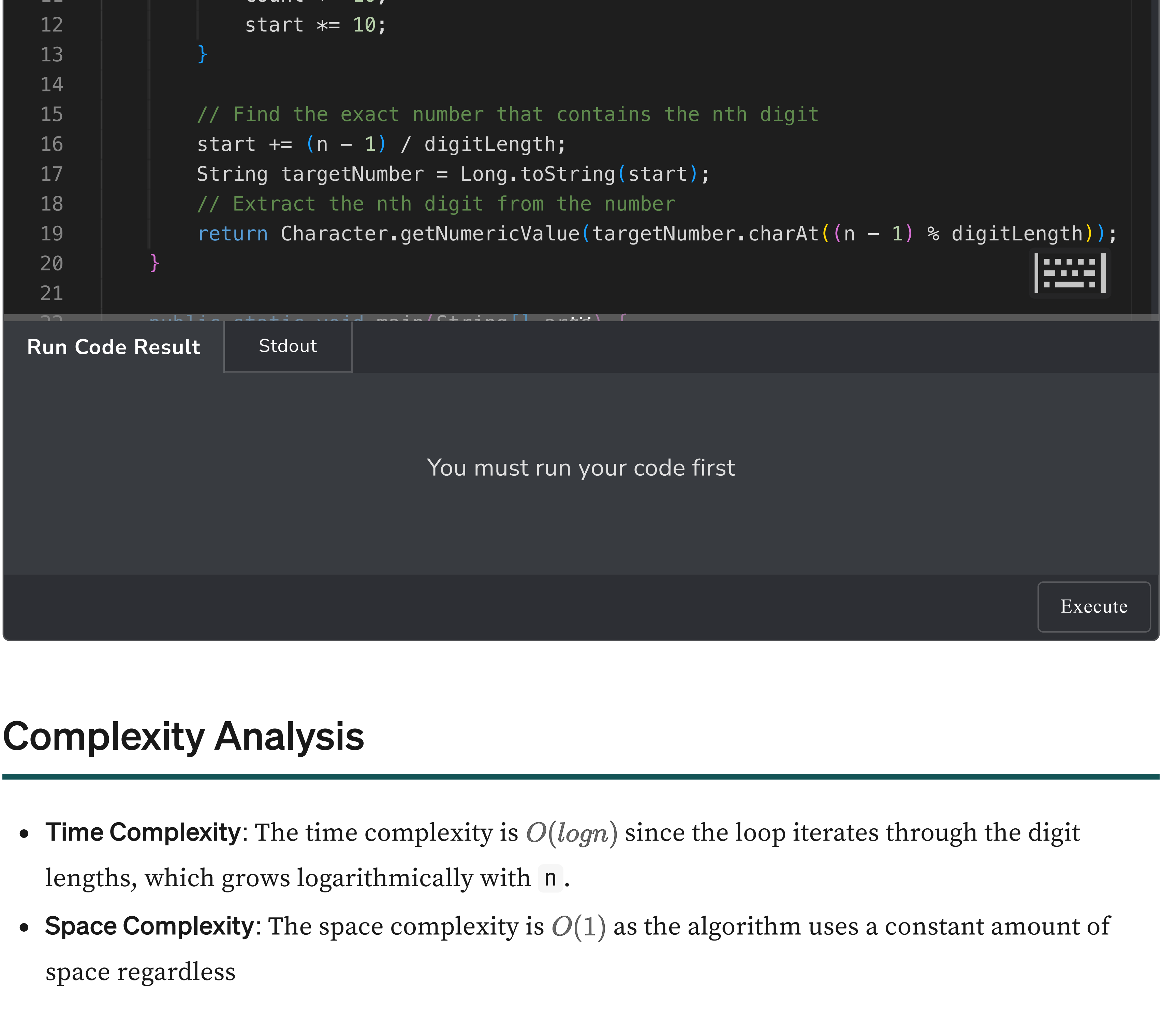
Step 5: Move to Five-Digit Numbers

- For the five-digit range (10000-99999), `n > digitLength ∗ count` is `71111 > 5 ∗ 90000` (False)
  - Now, we know the nth digit is within the five-digit numbers range.

Step 6: Find the Exact Number and Digit

- `start` is updated to locate the exact number containing the nth digit:
  - Update
    `start: start += (n − 1) / digitLength = 10000 + (71110 / 5) = 10000 + 14222 = 24222`
- The exact digit is at the `(5 − 1) % digitLength(5)`th position of `24222`, which is `2`.

Code



```java
public class Solution {
    public int findNthDigit(int n) {
        int digitLength = 1; // Starting with single-digit numbers
        long count = 9; // Count of digits for the current digit length
        long start = 1; // The starting number of the current digit length range

        // Find the digit length where the nth digit falls
        while (n > digitLength * count) {
            n -= digitLength * count;
            digitLength++;
            count *= 10;
            start *= 10;
        }

        // Find the exact number that contains the nth digit
        start += (n - 1) / digitLength;
        String targetNumber = Long.toString(start);
        // Extract the nth digit from the number
        return Character.getNumericValue(targetNumber.charAt((n - 1) % digitLength));
    }

```

Complexity Analysis

- **Time Complexity**: The time complexity is $O(logn)$ since the loop iterates through the digit lengths, which grows logarithmically with `n`.
- **Space Complexity**: The space complexity is $O(1)$ as the algorithm uses a constant amount of space regardless

← Previous
Nth Digit (Medium)

Next →
Restore IP Addresses (Medium)

☐ Mark as Completed